



Graph: Adjacency List Master Notes

Chai aur Code (Lecture 14) • Bachcho Jaisa Easy & Full Detailed Hinglish Edition 🚀



Python Graph DSA



Full Colorful Visuals



Deep Concept Link



Interview Cheat-Sheet

1. Graph Kya Hai? (Bachchon Jaisi Kahani) 🎈

Socho ek society mein 5 bachhe rehte hain: **1, 2, 3, 4 aur 5**. Sab aapas mein dost hain aur unke ghar aapas mein gully (edges) se jude hain.



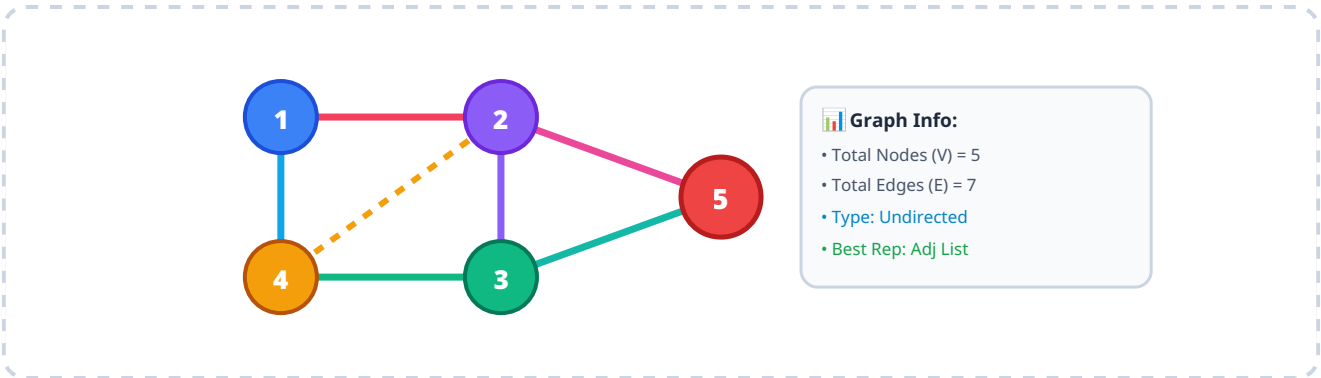
Chintu Ka Sawaal: "Bhaiya, agar hume computer ko batana ho ki kaun kiska dost hai aur kis raste se kahan ja sakte hain, to computer mein kaise likhenge?"

Computer ko ye samjhane ke **2 main tareeqe** hote hain:

- **Adjacency Matrix (Bada Board):** 5 bachho ke liye $5 \times 5 = 25$ dabbo ka bada chart banao. Sabhi mein 0 aur 1 bharo. *(Kharacha zyada, jagah ki barbadi!)*
- **Adjacency List (Dosti Diary):** Har bachhe ke naam ke aage ek choti diary laga do, jisme sirf uske **direct dosto** ke naam likhe hon! *(Sabse smart aur fast tareeqa!)*

2. Colorful Graph & List Representation Diagram 🌍

Neeche dekho video wala 5 Vertices aur 7 Edges ka graph aur uski memory list:



Adjacency List Mapping (Dosti Ki Complete Diary) 📖:

Bachha / Vertex (Source Key)	Connected Neighbours (List)	Degree (Kitne Connections?)
Node 1 ●	[2, 4]	Degree = 2 (2 dosto se juda hai)
Node 2 ●	[1, 3, 4]	Degree = 3 (3 dosto se juda hai)
Node 3 ●	[2, 4, 5]	Degree = 3 (3 dosto se juda hai)
Node 4 ●	[1, 2, 3, 5]	Degree = 4 (Sabse popular node!)
Node 5 ●	[3, 4]	Degree = 2 (2 dosto se juda hai)

3. Python Implementation: Step-by-Step Code

Python mein hum **Dictionary of Lists** ka use karte hain. C++/Java ke pointers ke muqable Python mein yeh bohot chota aur fast hota hai!

Secret Formula:

- `self.adj_list = {}` (Dictionary = Source Vertex ki keys)
- `self.adj_list[u].append(v)` (List = Uske saare neighbours)

```
class Graph:
    def __init__(self):
        # Step 1: Ek khali dictionary banai jo humari list of lists banegi
        self.adj_list = {}

    def add_vertex(self, vertex):
        # Step 2: Unique check - agar vertex pehle se nahi hai tabhi add karo
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []

    def add_edge(self, u, v, is_directed=False):
        # Step 3: Pehle ensure karo ki dono nodes exist karein
        if u not in self.adj_list:
            self.add_vertex(u)
        if v not in self.adj_list:
            self.add_vertex(v)

        # Step 4: Source se Destination ka connection jodo (u -> v)
        self.adj_list[u].append(v)

        # Step 5: Agar undirected graph hai, to wapsi ka rasta bhi jodo (v -> u)
        if not is_directed:
            self.adj_list[v].append(u)

    def print_graph(self):
        # Step 6: Pure graph ko sundar arrow format me print karo
        for vertex in self.adj_list:
            print(f"{vertex} -> {self.adj_list[vertex]}")

# =====
# Driver Code (Run karke Test karte hain)
# =====
g = Graph()

# 7 Edges add karte hain:
edges = [(1, 2), (1, 4), (2, 3), (2, 4), (3, 4), (3, 5), (4, 5)]
for u, v in edges:
    g.add_edge(u, v)

g.print_graph()
```

4. In-Degree aur Out-Degree (Social Media Example) 📱

📸 Instagram Analogy:

- **In-Degree (Followers):** Kitne log tumhare account par aa rahe hain (Incoming arrows).
- **Out-Degree (Following):** Tum kitne logo ko follow kar rahe ho (Outgoing arrows).
- **Undirected Graph:** Dono mutual best friends hain! Direct `len(adj_list[u])` hi degree hoti hai.

Type	Append Logic in Code	Total Elements Stored
Undirected Graph (2-Way)	<code>adj[u].append(v)</code> AND <code>adj[v].append(u)</code>	<code>2 * E</code> elements
Directed Graph (1-Way)	Sirf <code>adj[u].append(v)</code>	<code>E</code> elements

5. Matrix vs Adjacency List: Interview Comparison 🏆

Feature	Adjacency Matrix (Grid)	Adjacency List (Dict/Lists) ⭐
Space Complexity	<code>O(V²)</code> — Fixed 2D Grid	<code>O(V + E)</code> — No Memory Waste!
Best Use Case	Dense Graphs (Bohot zyada edges)	Sparse Graphs (95% real applications)
Add Naya Vertex	<code>O(V²)</code> (Poora grid bada karo)	<code>O(1)</code> (Bas ek nayi key banao)
Add Nayi Edge	<code>O(1)</code>	<code>O(1)</code>
Print All Neighbours	<code>O(V)</code> (Poori row scan karni padti hai)	<code>O(Degree(u))</code> (Direct list iterate karo)

🚀 Final Memory & Context Connection:

Adjacency List hi Graph DSA ki backbone hai! Isi dictionary structure par aage chal kar hum **BFS (Queue)** aur **DFS (Stack/Recursion)** ke traversal algorithms run karenge!